



Sandya Mannarswamy

Welcome to another instalment of 'CodeSport'. In this month's column, we will look at finding the inversion of a given permutation. We'll then discuss how to write efficient and correct code for multi-threaded applications.

Thanks to all the readers who had sent in their comments to the problems we discussed in the previous issue. Last month's takeaway problem was on finding the number of inversions in a permutation. Inversions form the basis for analysing the time complexity of sorting algorithms.

The inversion of a permutation is defined as follows:

- Permutation: 6 2 3 1 4 7 8 9 5
- Inversion: 3 1 1 1 4 0 0 0

Here, 1 has an inversion 3 as there are three elements greater than 1 that are left to 1 in the given permutation, and these are 6, 2, and 3. Similarly, 2 has an inversion 1 because there is only one element left to 2 which is greater than 2, and that is 6. The problem was to find the inversion of a permutation in the $O(n \log n)$ time complexity even at the cost of some memory.

Since I did not get any completely correct solutions, I am going to keep this question open for readers to respond to through this month also. Just to give you all a hint, recall the fact that a merge sort has a time complexity of $O(n \log n)$. Can the merge sort algorithm be modified to get the correct solution to the above question? Remember that a sorted array has no inversions.

This month's discussion

Most of the readers of LFY would know about the advent of multi-core processors and how they are likely to lead to a proliferation of multi-threaded software development. I had a few readers who had written in, requesting for a discussion on coding issues related to

writing multi-threaded code. Therefore, for the next three columns, we will be focusing on multi-threaded code, discussing some basic concepts of concurrency, what are the issues that a developer faces when writing multithreaded code, and what pitfalls he should try to avoid.

Why should I make my code multi-threaded?

You may ask why is there a sudden focus on multi-threaded code development. After all, as per Moore's law, processor speed doubles approximately every 18 months, thereby allowing the developer to take advantage of faster processors to speed up his applications. Well, the answer is that processor vendors are now finding that we have reached the limits of Moore's Law and it is no longer possible for the developers to depend on newer chips with faster clocks to speed up their applications. Why is that so? Due to thermal power and chip area restrictions, processor vendors are turning to multi-core processor designs, wherein each chip contains multiple cores, rather than speeding up the clock frequency of the processor itself. This in turn means that a single-threaded application running on one core of a newer multi-core processor can in fact run slower on that than on your existing and older non multi-core processor (which has a higher clock frequency). So if you are a software developer, to take advantage of these newer multi-core processors, you need to write multi-threaded code so that each thread can run on a separate core, thereby parallelising your application and making